Milestone 2 (M2) Report

Design and Implementation of an OFDM-Based Communication and Sensing Framework Using GNU Radio

GSoC category: open-source contributions to GNU Radio and SOTA positioning

Name	Vamsi Pancholi
Project ID	G17, g17_gsoc_isac_ind (individual)
Course	ECE 310, Open-Source Contribution Track (GNU Radio / GSoC, ISAC domain)
Target organization	GNU Radio
Repository	https://github.com/vamsip-cell/g17_gsoc_isac_ind
M2 contributions	PR #8231 (closes issue #7064); investigation of issue #7997

1. Summary of M2 Progress

M2 is about getting into the GNU Radio codebase and its contribution process. Two pieces of work were completed:

1. **Pull request #8231** to *gnuradio/gnuradio*, which closes issue #7064. It rewrites the GNU Radio Companion (GRC) documentation of the Options block, which was outdated and partly wrong. The PR is open, its DCO sign-off check has passed, and it is awaiting maintainer review.
2. **An investigation of issue #7997**, which reports that inserting a Channel Model into an OFDM flowgraph freezes the whole flowgraph. A reproduction script was written and run. It shows that the Channel Model alone does not cause the freeze, and it identifies two other configurations that produce the same symptom.

The second item is directly about GNU Radio's OFDM chain, which the planned sensing block will attach to.

2. Contribution 1: Pull Request #8231 (Issue #7064)

Pull request	#8231, "grc: rewrite Options block documentation to match current parameters"
Closes	Issue #7064, "GRC: Revise Options Block YML documentation section" (opened Jan 2024)
Change	1 file (*grc/core/blocks/options.py*), +37 / −8 lines, 1 signed-off commit
Opened	11 Oct 2026, from *vamsip-cell:grc-options-docs* into *gnuradio:main*
Status	Open. DCO check passed. Full CI awaiting maintainer approval (standard for first-time contributors). No review yet.

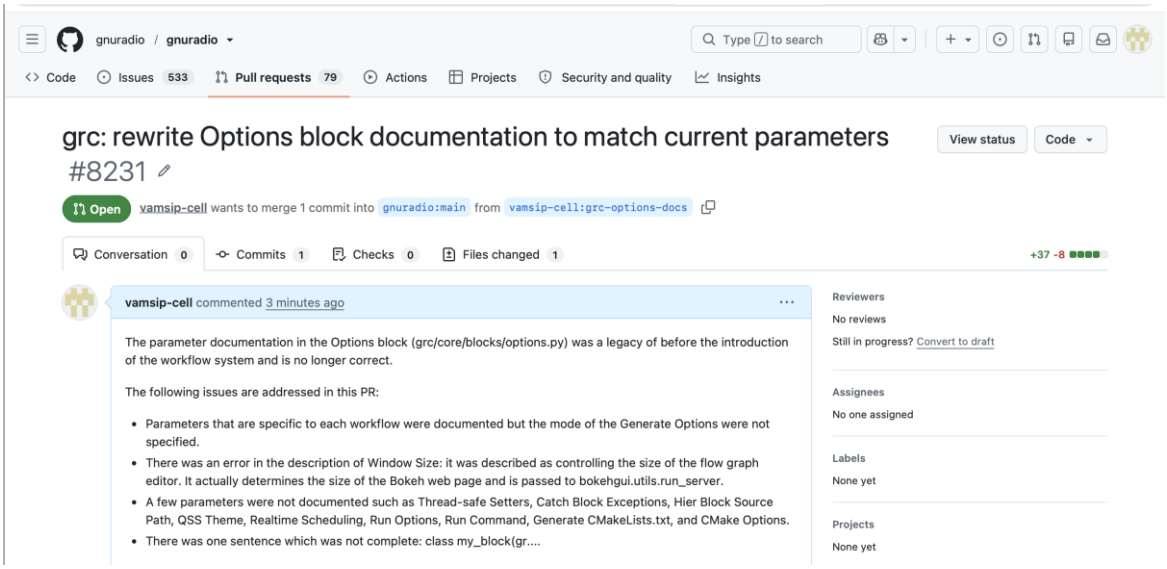


Figure 1: Pull request #8231 on GitHub (title, Open status, gnuradio:main ← vamsip-cell:grc-options-docs)

2.1 What the problem was

Every GRC flowgraph has exactly one Options block, which sets the flowgraph's title, generated language, application type, and runtime options. Its Documentation tab is what users read to understand these settings. Issue #7064 asked for that documentation to be updated, for the Bokeh-only parameters to be grouped and marked, and for the parameters to be listed in order.

The issue pointed at *options.block.yml*, but in the current *main* branch that file is only a stub. The Options block is now defined in Python (*grc/core/blocks/options.py*), and most of its parameters come from per-mode "workflow" files (*grc/blocks/*.workflow.yml*). Each Generate Options mode (QT GUI, Bokeh GUI, No GUI, Hier Block, Hier Block (QT GUI), in Python or C++) adds its own parameters. Comparing the documentation string against these files showed four problems:

- Mode-specific parameters (Window size, Run, Category, Max Number of Output) were described without saying which mode they apply to.
- Window size was described as controlling "the dimensions of the flow graph editor". It actually sets the size of the Bokeh web page: the generated code passes it to *bokehgui.utils.run_server*.
- Nine visible parameters were not documented at all: Thread-safe setters, Catch Block Exceptions, Hier Block Source Path, QSS Theme, Realtime Scheduling, Run Options, Run Command, Generate CMakeLists.txt and CMake options.
- One sentence was cut off mid-word ("class my_block(gr....").

2.2 What the PR changes

The documentation is rewritten and grouped by mode: parameters available in every mode, the Advanced tab, parameters shared by the GUI modes, No-GUI-only, Bokeh-only, QT-only, Hier-Block-only and C++-only. Each description was checked against the code that uses the parameter, for example:

- **Max Number of Output** is passed to *tb.start(max_nouts)*, and it caps how many items a block may produce per call to *work()*, which bounds latency (0 means no limit).

- **Catch Block Exceptions** is passed to the top block. *thread_body_wrapper.h* shows that when it is on, an exception in a block's thread is caught and logged ("ERROR thread[...]") and that thread stops. When off, the exception terminates the program.
- **Run Options** (No GUI) decides whether the generated program runs to completion or waits for Enter, which is visible directly in the No GUI code template.

The hidden Linking parameter is not documented, since it is not shown in the dialog. Moving GUI-only parameters into a separate tab, suggested in the issue thread, is left for a follow-up because it changes GRC's interface and needs maintainer agreement first.



```
diff --git a/grc/core/blocks/options.py b/grc/core/blocks/options.py
index b04638a..80433ae 100644
--- a/grc/core/blocks/options.py
+++ b/grc/core/blocks/options.py
@@ -174,14 +174,43 @@ class Options(Block):

         self.cpp_templates = MakoTemplates()
         self.documentation['documentation'] = """
-            The options block sets special parameters for the flow graph. Only one option block is allowed per flow graph.
-            Title, author, and description parameters are for identification purposes.
-            The window size controls the dimensions of the flow graph editor. The window size (width, height) must be between (300, 300) and (4096, 4096).
-            The generate options controls the type of code generated. Non-graphical flow graphs should avoid using graphical sinks or graphical variable controls.
-            In a graphical application, run can be controlled by a variable to start and stop the flowgraph at runtime.
-            The id of this block determines the name of the generated file and the name of the class. For example, an id of my_block will generate the file my_block
-            The category parameter determines the placement of the block in the block selection window. The category only applies when creating hier blocks. To put
-            The Max Number of Output is the maximum number of output items allowed for any block in the flowgraph; to disable this set the max_nouts equal to 0.Use
+            The Options block sets flow graph wide parameters and controls code generation. Only one Options block is allowed per flow graph.
+            The parameters shown depend on the selected Output Language and Generate Options.
+
+            Parameters available in every mode:
+            Id: determines the name of the generated file and of the generated class. For example, an id of my_block generates the file my_block.py and the class my
+            Title, Author, Copyright, Description: identify the flow graph and are written into the header of the generated code. The title is also used as the wind
+            Output Language: Python or C++.
+            Generate Options: the type of application generated (QT GUI, Bokeh GUI, No GUI, Hier Block or Hier Block (QT GUI)). The available choices depend on the
+
+            Advanced tab (every mode):
+            Thread-safe setters: when On, the generated setter methods hold a lock, so variables can be changed safely from other threads.
+            Catch Block Exceptions: when On, an exception raised in a block's thread is caught and logged, and that block's thread stops. When Off, the exception is
+            Hier Block Source Path: paths searched for the .grc source files of hier blocks used in this flow graph.
```

Figure 2: The change submitted in PR #8231 (diff of grc/core/blocks/options.py): old documentation in red, new in green

2.3 Why it is technically meaningful

The documentation is the only in-tool explanation of options that every GRC user sees. Two of the errors could actively mislead: a user changing Window size expecting the editor to resize, or a user debugging a flowgraph without knowing that Catch Block Exceptions is what turns a crashing block into a logged error and a silent stall. The second point connects directly to Contribution 2, where exactly this behavior (a logged *ERROR thread[...header_payload_demux...]* followed by a frozen flowgraph) was observed.

2.4 How it was verified

- The patch applies cleanly to current *main*, and passes the *pycodestyle* check that GNU Radio's CI runs (max line length 120, CI ignore list).
- GRC's own *grc/tests/test_generator.py* passes with the change.
- The Bokeh GUI test flowgraph was loaded and its visible Options parameters were compared with the new text.
- Open PR #8214 also edits *options.py* (it changes the visibility of the ID parameter). The two changes do not overlap, and GitHub reports that #8231 merges cleanly. This is noted in the PR description.

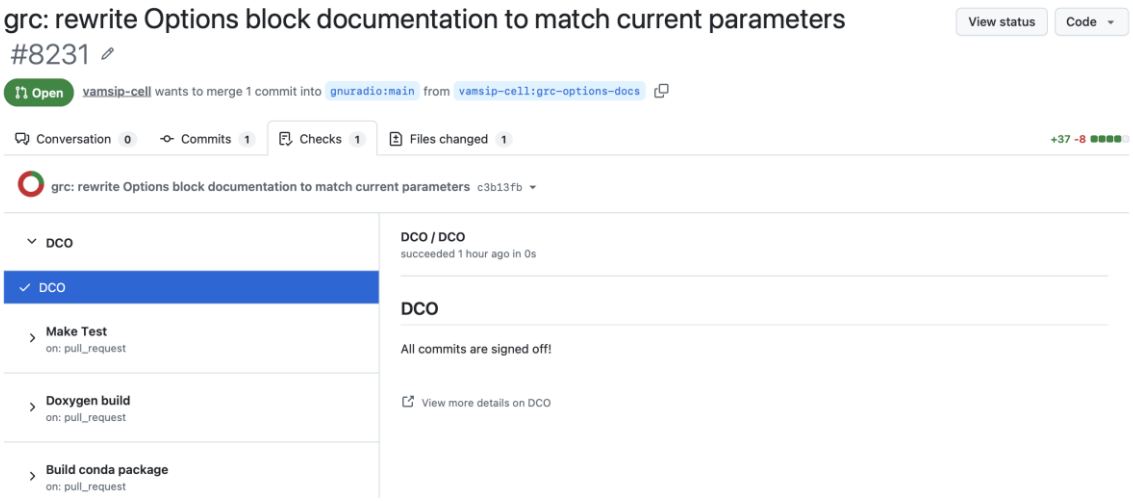


Figure 3: Checks tab of PR #8231 showing the DCO check passed

3. Contribution 2: Investigation of Issue #7997

3.1 The reported problem

Issue #7997 (opened Jan 2026, GNU Radio 3.10.11, Ubuntu) reports that adding a Channel Model block between the transmitter's IFFT and the receiver's FFT in an OFDM flowgraph freezes the entire flowgraph, including the source, and that removing the channel restores normal operation. The reporter's flowgraph includes a custom *gr.basic_block* pilot inserter that uses *general_work()* and *consume_each()* on vector ports. The flowgraph file and the block's code were not attached, and the issue had no response for nine months.

3.2 Method

A script (*M2_G17/Code/src/repro_7997.py*) builds a set of flowgraphs and runs each one in its own process with a 30-second timeout, so a frozen case cannot block the test. Three groups of cases are tested:

- **Stock OFDM chain:** *ofdm_tx* → (Channel Model) → *ofdm_rx*, with the same parameters as GNU Radio's *ofdm_loopback.grc* example (64-point FFT, cyclic prefix 16, 8 data and 4 pilot carriers, 50-byte packets). Because the OFDM receiver never signals completion, each case runs for 6 s and counts correctly decoded packets. A case is marked STALL if no data arrived in the last 3 s.
- **Custom chain like the reporter's:** QPSK symbols → custom pilot inserter (*gr.basic_block*, *general_work()* + *consume_each()*, 48 data and 4 pilot carriers) → IFFT → (Channel Model) → FFT, with a finite source. OK if the flowgraph finishes.
- **Deliberately faulty block:** the same chain, but the pilot inserter waits for 1000 input vectors while its *forecast()* still declares a 1:1 ratio.

The varied settings are the presence of the channel, its timing offset (epsilon), whether the channel blocks stream tags, and the receiver's length-tag key.

3.3 Results

Ubuntu 24.04, GNU Radio 3.10.9.2 (full output in *M2_G17/Results/repro_7997_results_ubuntu.txt*):

Case	Configuration	Result
stock_no_channel	No channel	OK: 248 packets, all correct
stock_channel_eps1	Channel, epsilon = 1	OK: 248 packets, all correct
stock_channel_eps1.0001	Channel, timing offset	OK: 247 packets, all correct
stock_channel_tags_pass	Channel passes tags	OK: 248 packets, all correct
stock_channel_same_key	Same tag key, channel blocks tags	OK: 247 packets, all correct
stock_nochan_same_key	Same tag key, tags reach RX	STALL: 0 packets, "Buffer too small for min_noutput_items"
stock_chan_tagspass_same_key	Same tag key, channel passes tags	STALL: 0 packets, same error
custom_no_channel	Custom pilot inserter	OK: 2000 of 2000 symbols
custom_channel_eps1 / _eps1.0001	Custom + channel	OK: 1999 of 2000 symbols
fragile_no_channel / _channel_eps1	Forecast mismatch	HANG (with and without channel)

On macOS (GNU Radio 3.10 from conda-forge), GNU Radio's *ofdm_loopback.grc* example, which contains a Channel Model, was run in GRC and decoded packets normally (Figure 4). The one missing symbol in the custom channel cases is not a freeze: the Channel Model's fractional resampler holds back a few samples at the end of a finite stream, so the last OFDM symbol is incomplete.

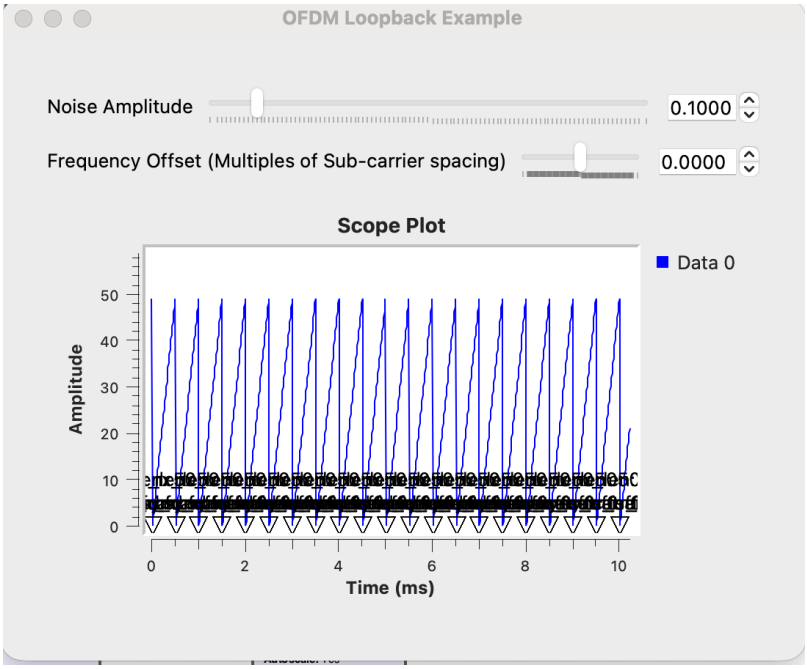


Figure 4: macOS: GRC ofdm_loopback example with a Channel Model, decoding packets normally (each ramp 0 to 49 is one correctly decoded packet)

3.4 Interpretation

The Channel Model never caused a freeze in any test. The same total-freeze symptom does occur in two configurations that do not depend on it:

1. **Length-tag key collision.** *ofdm_tx* marks packet boundaries with a *packet_len* tag. If *ofdm_rx* is configured to use the same key for its own output and those transmitter tags reach it, its internal *header_payload_demux* block fails with *Buffer too small for min_noutput_items*. With Catch Block Exceptions on, that block's thread stops after logging the error. Its input buffer then fills up, and backpressure stops every block upstream, including the source. GNU Radio's own example avoids this by using a separate key (*rx_len*) for the receiver. A Channel Model with tag blocking turned on prevents the collision, which is why the same-key case with tag blocking ran normally.
2. **Forecast and consume mismatch.** A *general_work()* block that returns 0 until it has more input than its *forecast()* declares is never given that input, because the scheduler only provides what the forecast asks for, so the graph hangs. This matches the reporter's description of a custom *gr.basic_block* with explicit *consume_each()*.

The accurate status of the issue is therefore "not reproducible with stock blocks or a correctly written custom block". The reported freeze is most likely caused by the reporter's custom block or tag-key setup, but confirming which one requires their flowgraph. The next step on the issue is to share these results and the script with the reporter and ask for their flowgraph and block code.

3.5 Limitations

- The reporter's exact flowgraph and block code are unavailable, so their freeze could not be reproduced directly.
- The script was run on GNU Radio 3.10.9.2. The reporter used 3.10.11, which is close but not identical.
- All tests are simulation-only, with no SDR hardware.

4. Maintainer and Community Response

Item	Interaction	Response so far
Issue #7064	Comment claiming the issue (11 Oct 2026), tagging the issue author	None yet. GitHub links PR #8231 on the issue.
PR #8231	Opened 11 Oct 2026	DCO check passed. CI and review pending maintainer action.
Issue #7997	Comment announcing the investigation (9 Oct 2026). Findings are documented in this report and the repository; sharing them on the issue is a planned next step.	No reply from the reporter yet.

No maintainer response has been received at the time of writing. Both contributions were made within the last few days, and GNU Radio reviews by volunteer maintainers commonly take days to weeks. Any responses will be reported in M3.

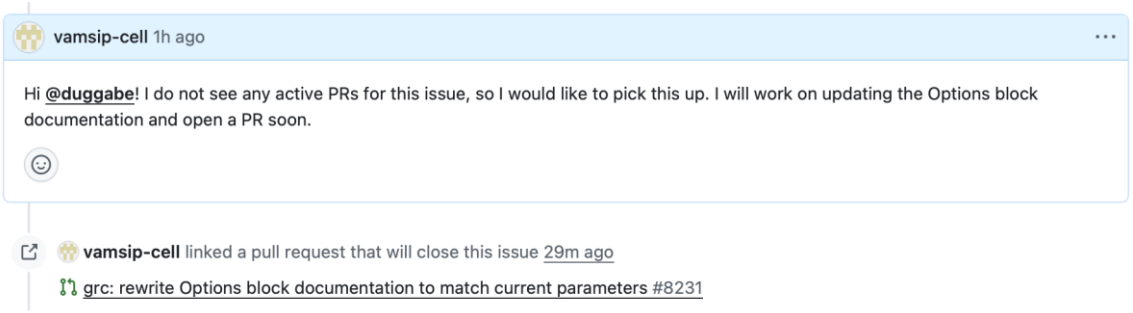


Figure 5: Issue #7064: claim comment and the linked pull request #8231

5. SOTA Positioning in the GNU Radio Ecosystem

Tool	Communication	Sensing	Relevant limitation
gr-digital (in-tree)	Yes: OFDM TX/RX	No	Channel estimate used internally only, once per frame; not exposed at the receiver output.
gr-radar (OOT)	No	Yes: OFDM radar and others	Separate radar flowgraphs; does not decode data.
gr-ofdmradar (OOT)	No (noted as possible, not implemented)	Yes: OFDM range-Doppler	Sensing only, separate from gr-digital's receiver.
This project (planned)	Yes: reuses gr-digital	Yes: on the same frames	Single antenna, single node, simulation first.

OFDM radar itself is already available in the GNU Radio ecosystem through two out-of-tree modules. **gr-radar** (KIT CEL) is a radar toolbox with example flowgraphs for CW, FSK, FMCW and OFDM radar, partly tested on USRP hardware. **gr-ofdmradar** (Analog Devices) is an OFDM radar transmitter and receiver with range-Doppler visualization; its README notes that parts of the waveform could carry data, but that this is not implemented. Both are sensing-only, while gr-digital is communication-only. The gap this project addresses is therefore precise: no GNU Radio implementation decodes data and estimates range and Doppler from the same OFDM frames.

The method remains the one chosen in M1: reciprocal filtering (dividing each received symbol by the known transmitted symbol) followed by a 2D FFT over subcarriers and OFDM symbols (Sturm and Wiesbeck, 2011), which Braun (2014) analyzes in detail and gr-ofdmradar builds on. What is new in this project is not the algorithm but where it runs: inside a working gr-digital OFDM link, so that one set of frames serves both data decoding and range-Doppler estimation.

The M2 work also gave a practical result for that design. The sensing block will sit next to *ofdm_rx* and handle the same stream tags, so it must not reuse the transmitter's length-tag key (Section 3.4) and must declare its input needs correctly in *forecast()*. Both failure modes found in #7997 are concrete design constraints for the block.

6. Next Planned Contributions

1. **Follow up on PR #8231:** respond to review comments, and, if maintainers agree, move the GUI-only parameters into a separate tab as suggested in issue #7064.
2. **Report the #7997 findings on the issue:** post the results and a link to the script, and ask the reporter for their flowgraph and pilot inserter code.
3. **Small follow-up from #7997:** propose on the issue either documenting in *ofdm_rx* that the receiver's length-tag key must differ from the transmitter's, or making the *Buffer too small* error explain the likely cause. This will be proposed before any PR, since it is a design decision for maintainers.
4. **Main project work:** an out-of-tree module with an OFDM sensing block that takes the receiver's frames and the known transmitted symbols, performs reciprocal filtering and a 2D FFT, and outputs a range-Doppler map. It will be validated first against a simulated point target with known range and velocity.

7. Updated Timeline

Phase	Dates	Milestone	Activities
Foundation	Sep 15 to Sep 28, 2026	M1	Problem, literature review, SOTA (done).
Upstream contribution	Sep 29 to Oct 11, 2026	M2	PR #8231, #7997 investigation, SOTA refinement (done).
Sensing block	Oct 12 to Nov 1, 2026	M3	OOT module; reciprocal filter and 2D FFT; simulated target tests.
Integration and validation	Nov 2 to Nov 22, 2026	M3	Attach to gr-digital OFDM RX; KPIs: BER, range and velocity error.
Upstream follow-up and finalization	Nov 23 to Dec 7, 2026	Final	Address PR review; sidelobe suppression (windowing); documentation.

8. Use of Generative AI

An AI assistant (Claude) was used to help find suitable issues, read the GNU Radio source, draft the reproduction script and the documentation text, and draft this report. All results reported here come from running the script and the GRC example, and the technical claims were checked against the GNU Radio source code. I have reviewed the submitted work and am responsible for its content.

References

[1] C. Sturm and W. Wiesbeck, "Waveform Design and Signal Processing Aspects for Fusion of Wireless Communications and Radar Sensing," Proc. IEEE, vol. 99, no. 7, pp. 1236–1259, 2011, doi: 10.1109/JPROC.2011.2131110.

[2] M. Braun, "OFDM Radar Algorithms in Mobile Communication Networks," Ph.D. dissertation, Karlsruhe Institute of Technology, 2014, doi: 10.5445/IR/1000038892.

[3] M. Mirabella, P. D. Viesti, A. Davoli, and G. M. Vitetta, "Deterministic Signal Processing Techniques for OFDM-Based Radar Sensing: An Overview," IEEE Access, vol. 11, pp. 68872–68889, 2023, doi: 10.1109/ACCESS.2023.3292937.

[4] gr-radar: GNU Radio Radar Toolbox. https://github.com/kit-cel/gr-radar

[5] gr-ofdmradar: OFDM radar for GNU Radio (Analog Devices). https://github.com/analogdevicesinc/gr-ofdmradar

[6] GNU Radio, gr-digital/python/digital/ofdm_txrx.py. https://github.com/gnuradio/gnuradio/blob/main/gr-digital/python/digital/ofdm_txrx.py

[7] GNU Radio pull request #8231. https://github.com/gnuradio/gnuradio/pull/8231

[8] GNU Radio issue #7064. https://github.com/gnuradio/gnuradio/issues/7064

[9] GNU Radio issue #7997. https://github.com/gnuradio/gnuradio/issues/7997

[10] Project repository, M2_G17 folder. https://github.com/vamsip-cell/g17_gsoc_isac_ind/tree/main/M2_G17